HUST

TRƯỜNG ĐẠI HỌC BÁCH KHOA HÀ NỘI

HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

ONE LOVE. ONE FUTURE.

TRƯỜNG ĐẠI HỌC
BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY
OF SCIENCE AND TECHNOLOGY

LÝ THUYẾT MẬT MÃ

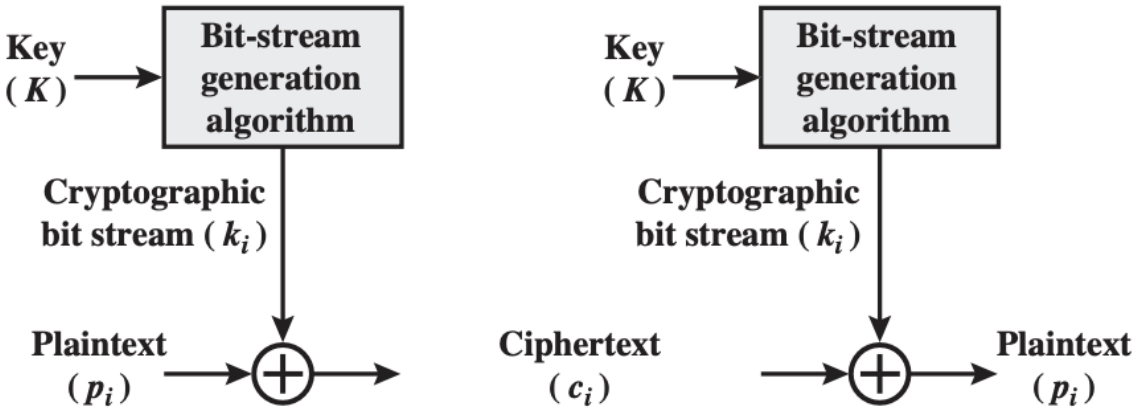
TS. TẠ KIM HUỆ
TRƯỜNG ĐIỆN – ĐIỆN TỬ

ONE LOVE. ONE FUTURE.

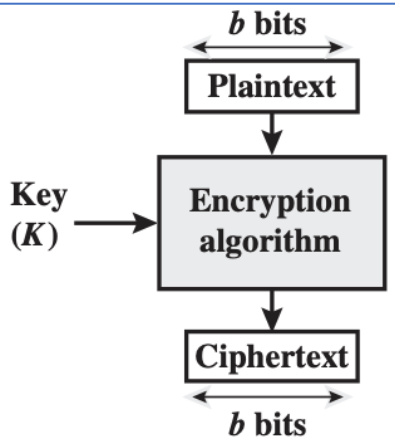
CHƯƠNG 2: MẬT MÃ KHOÁ ĐỐI XỨNG

- Mật mã học hiện đại và các khái niệm
- Mật mã dòng
- **Mật mã khối**

Sự khác nhau của mật mã dòng và mật mã khối



(a) Stream cipher using algorithmic bit-stream generator



(b) Block cipher

Tóm tắt: PRPs and PRFs

- Pseudo Random Function (**PRF**) định nghĩa trên tập (K,X,Y):

$$F: K \times X \rightarrow Y$$

là một thuật toán hiệu quả "efficient" algorithm để ước lượng F(k,x)

- Pseudo Random Permutation (**PRP**) định nghĩa trên tập (K,X):

$$E: K \times X \rightarrow X$$

thỏa mãn:

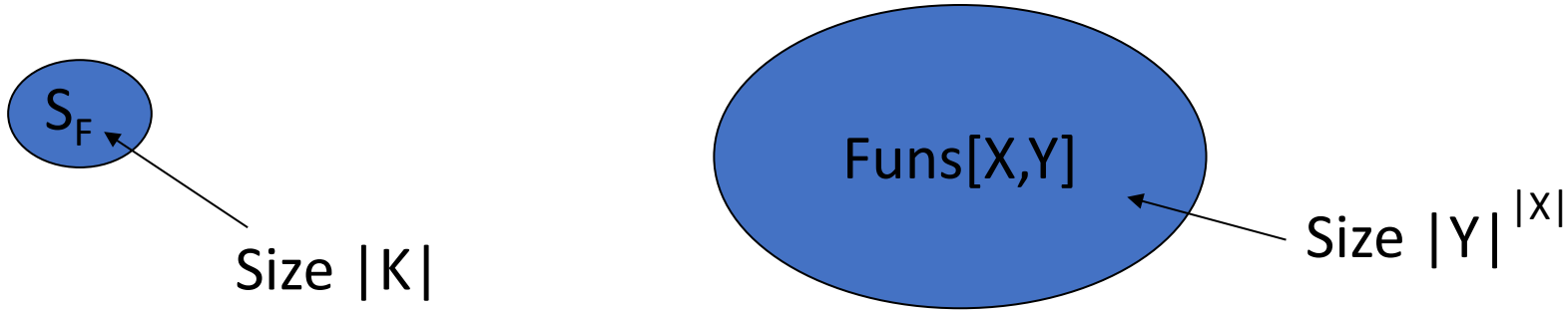
1. Tồn tại "efficient" <u>deterministic</u> algorithm để ước lượng E(k,x)
2. Hàm $E(k, \cdot)$ là một song ánh (one-to-one)
3. Tồn tại "efficient" inversion algorithm D(k,x)

Secure PRFs

- Cho $F: K \times X \rightarrow Y$ là một PRF

$$\begin{cases} \text{Funs}[X,Y]: \text{ là tập các hàm từ } X \text{ thành } Y \\ S_F = \{ F(k,\cdot) \text{ s.t. } k \in K \} \subseteq \text{Funs}[X,Y] \end{cases}$$

- <u>Intuition</u>: a PRF is **secure** if
  Một hàm ngẫu nhiên trong Funs[X,Y] là không thể phân biệt với một hàm ngẫu nhiên trong S_F

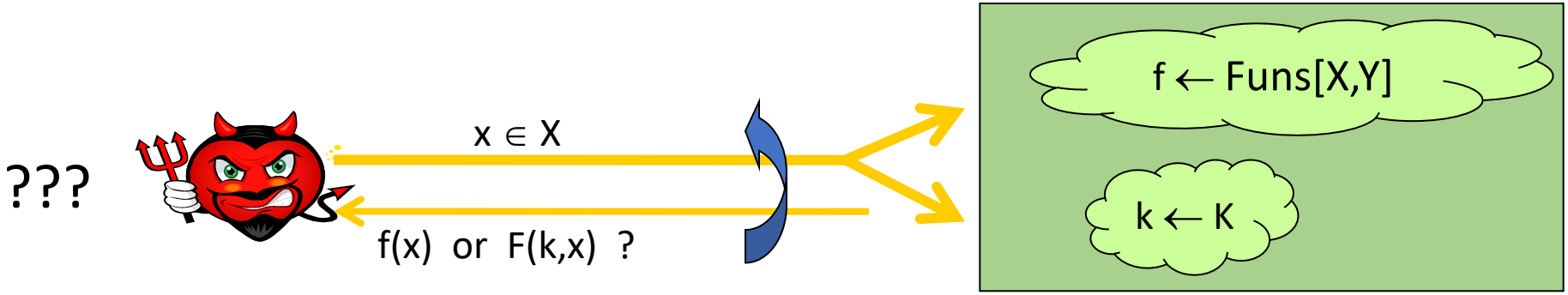


Secure PRFs

- Cho $F: K \times X \rightarrow Y$ là một PRF

$$\begin{cases} \text{Funs}[X,Y]: & \text{là tập các hàm từ X đến Y} \\ S_F = \{ F(k,\cdot) \text{ s.t. } k \in K \} & \subseteq \text{Funs}[X,Y] \end{cases}$$

- <u>Trực giác (Intuition)</u>: một hàm PRF là bảo mật nếu một hàm ngẫu nhiên trong Funs[X,Y] là không phân biệt được với một hàm ngẫu nhiên trong S_F



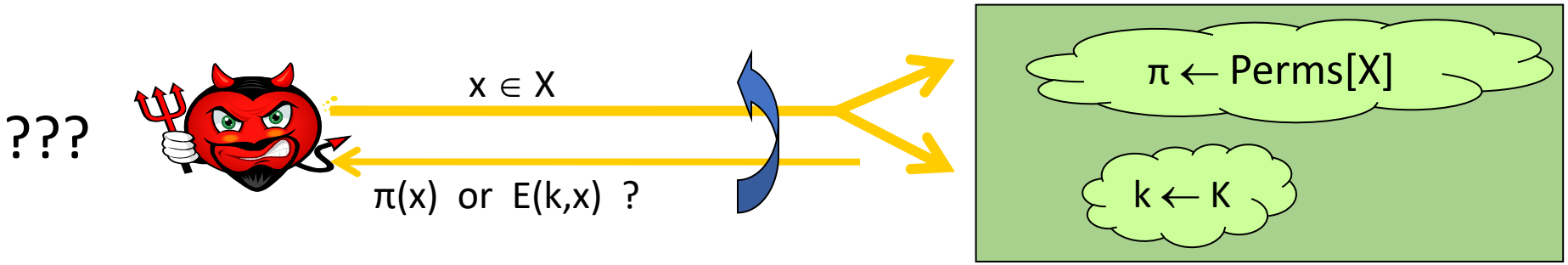
Secure PRPs (secure block cipher)

- Cho $E: K \times X \rightarrow X$ là một PRP

Perms[X]: là tập các song ánh (**one-to-one**) từ X thành X

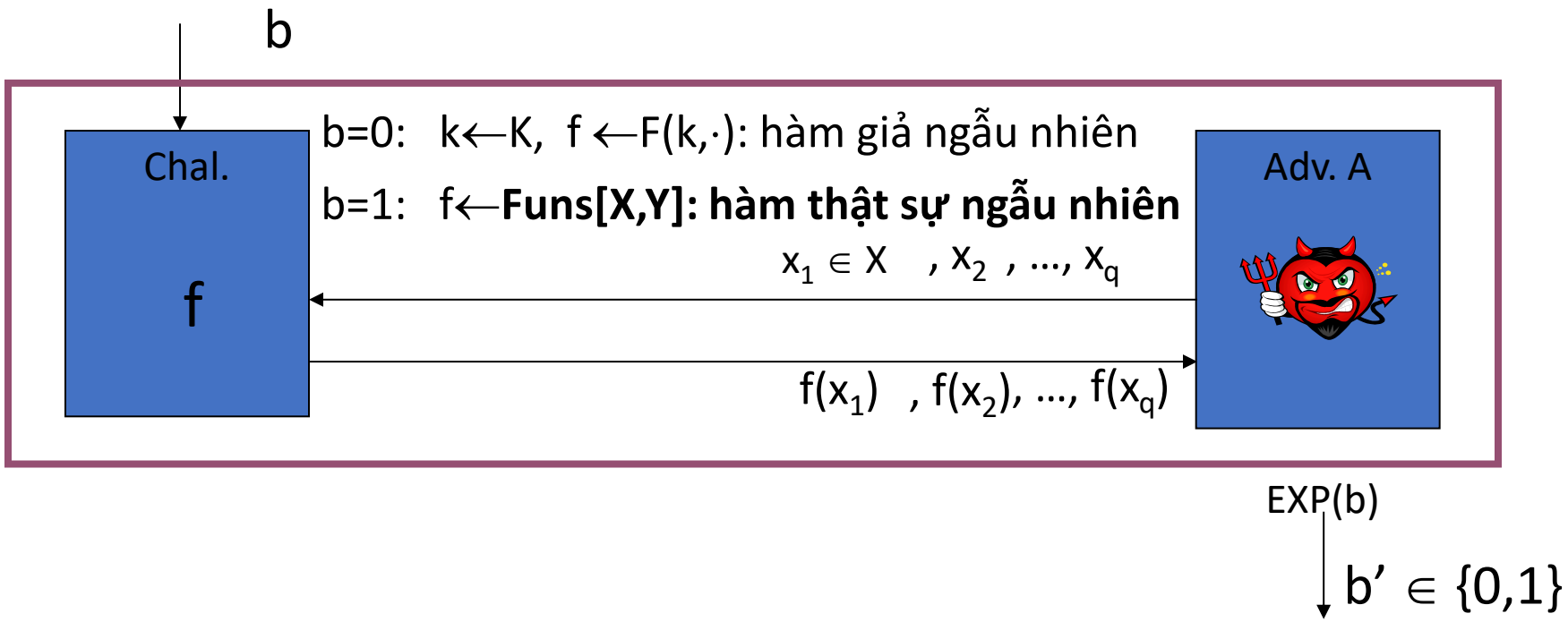
$S_F = \{ E(k,\cdot) \text{ s.t. } k \in K \} \subseteq \text{Perms}[X]$

- Trực giác (Intuition): một PRP là bảo mật nếu một hàm ngẫu nhiên trong Perms[X] là không thể phân biệt với một hàm ngẫu nhiên trong S_F



Secure PRF: definition

- Cho b=0,1 định nghĩa phép kiểm định EXP(b) như sau:



- Định nghĩa: F is a secure PRF if for all "efficient" A:

$$Adv_{PRF}[A,F] := |Pr[EXP(0)=1] - Pr[EXP(1)=1]|$$

is "negligible."

Secure PRF: definition

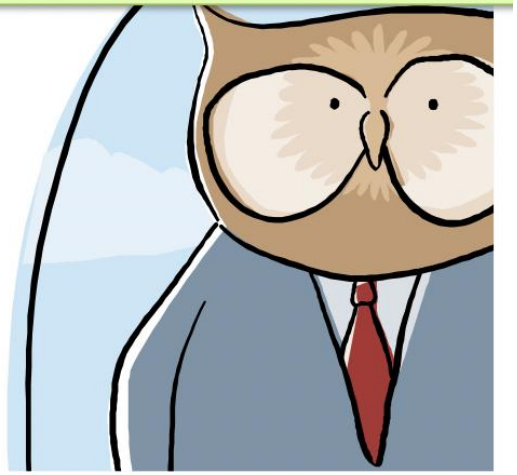
security parameter
n

Scenario 0 : EXP(0)

distinguisher D

Scenario 1 : EXP(1)

challenger
chooses a random $k \in \{0,1\}^n$.



$x_1 \in \{0,1\}^u$ →

← $F_k(x_1)$

$x_2 \in \{0,1\}^u$ →

← $F_k(x_2)$

⋮

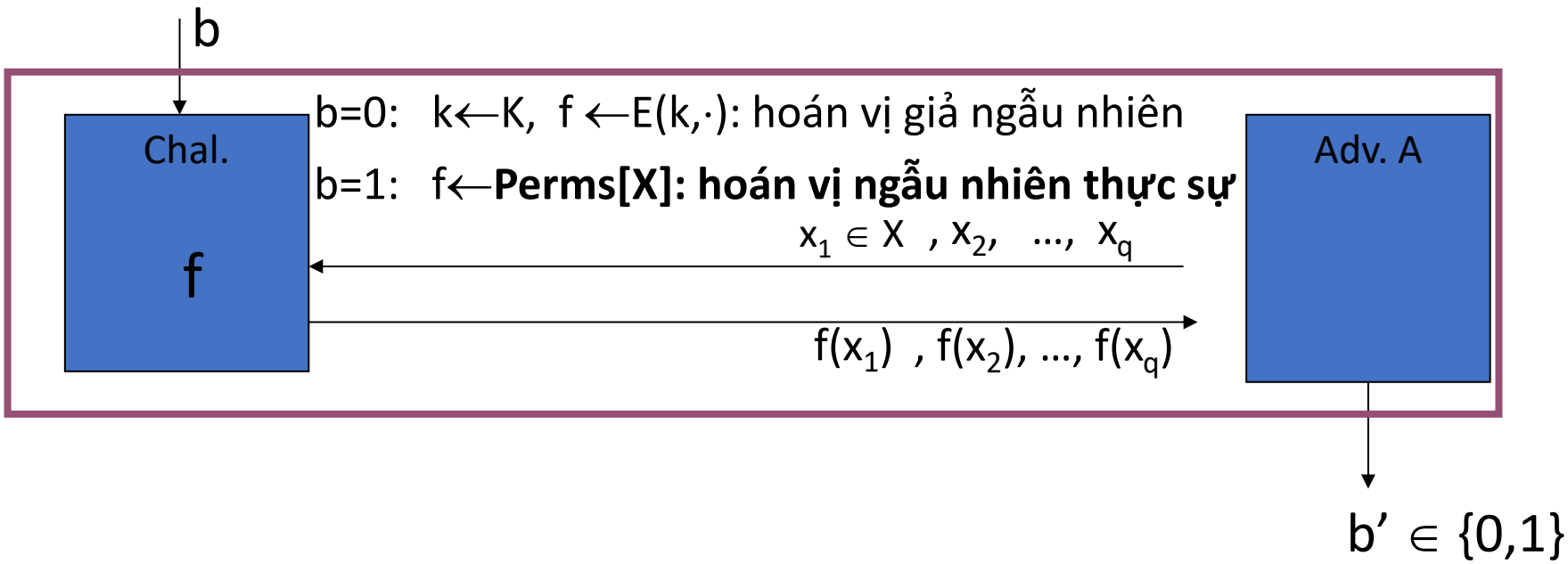
$x_t \in \{0,1\}^u$ →

← $F_k(x_t)$

$$\mathrm{Adv}_{PRF}[A,F] := \left| \Pr[EXP(0)=1] - \Pr[EXP(1)=1] \right|$$

Secure PRP (secure block cipher)

- For b=0,1 define experiment EXP(b) as:



- Def: E is a secure PRP if for all "efficient" A:

$$Adv_{PRP}[A,E] = |Pr[EXP(0)=1] - Pr[EXP(1)=1]|$$

is "negligible."

Example secure PRPs

- Cho X = {0,1}. Perms[X] chứa 2 hàm
- Xem xét PRP sau đây: key space K={0,1}, input space X = {0,1}, PRP defined as: $E(k,x) = x \oplus k$
- Đây có phải PRP bảo mật ?

→ Yes

No

It depends

Attacker A:

(1) query f(·) at x=0 and x=1

$Adv_{PRF}[A,E] = |1/2 - 1/2| = 0$

Example secure PRPs

- <u>PRPs believed to be secure</u>: 3DES, AES, …

AES-128: $K \times X \rightarrow X$ where $K = X = \{0,1\}^{128}$

- An example concrete assumption about AES:

All 2^{80}–time algs.

A have $\mathrm{Adv}_{\mathrm{PRP}}[A, \mathbf{AES}] < 2^{-40}$

- Consider the 1-bit PRP from the previous question: $E(k,x) = x \oplus k$
- Is it a secure PRF?
- Note that Funs[X,X] contains four functions

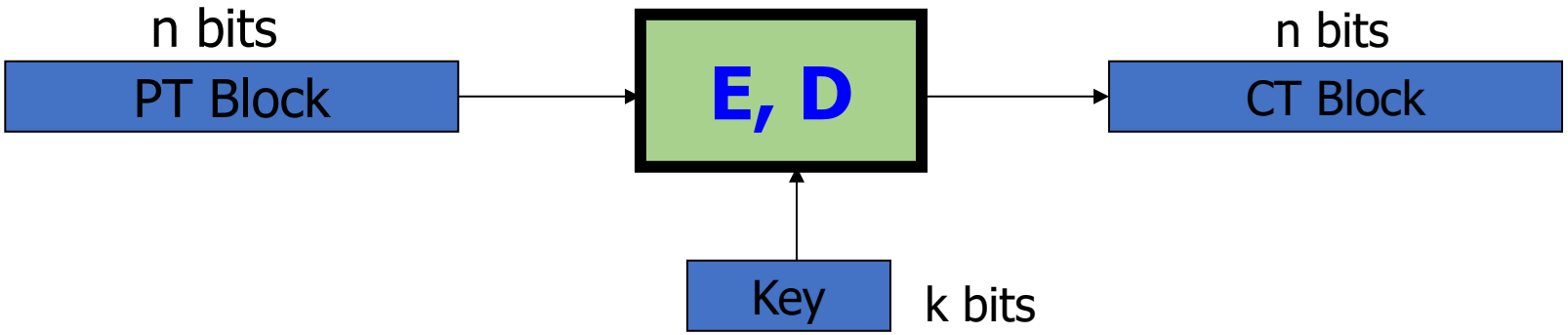
Yes

No

It depends

Attacker A:
(1) query $f(\cdot)$ at $x=0$ and $x=1$
(2) if $f(0) = f(1)$ output "1", else "0"
$Adv_{PRF}[A,E] = |0-½| = ½$

Block ciphers: crypto work horse



Canonical examples:

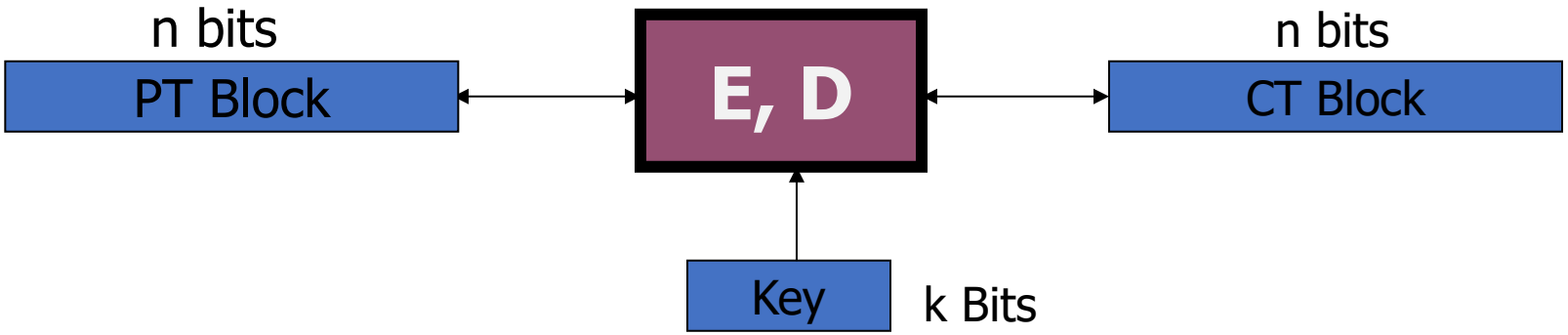
1. 3DES: n= 64 bits, k = 168 bits
2. AES: n=128 bits, k = 128, 192, 256 bits

Hiệu năng (Performance)

AMD Opteron, 2.2 GHz (Linux)

	Cipher	Block/key size	Speed (MB/sec)
stream	RC4		126
	Salsa20/12		643
	Sosemanuk		727
block	3DES	64/168	13
	AES-128	128/128	109

Block ciphers: crypto work horse



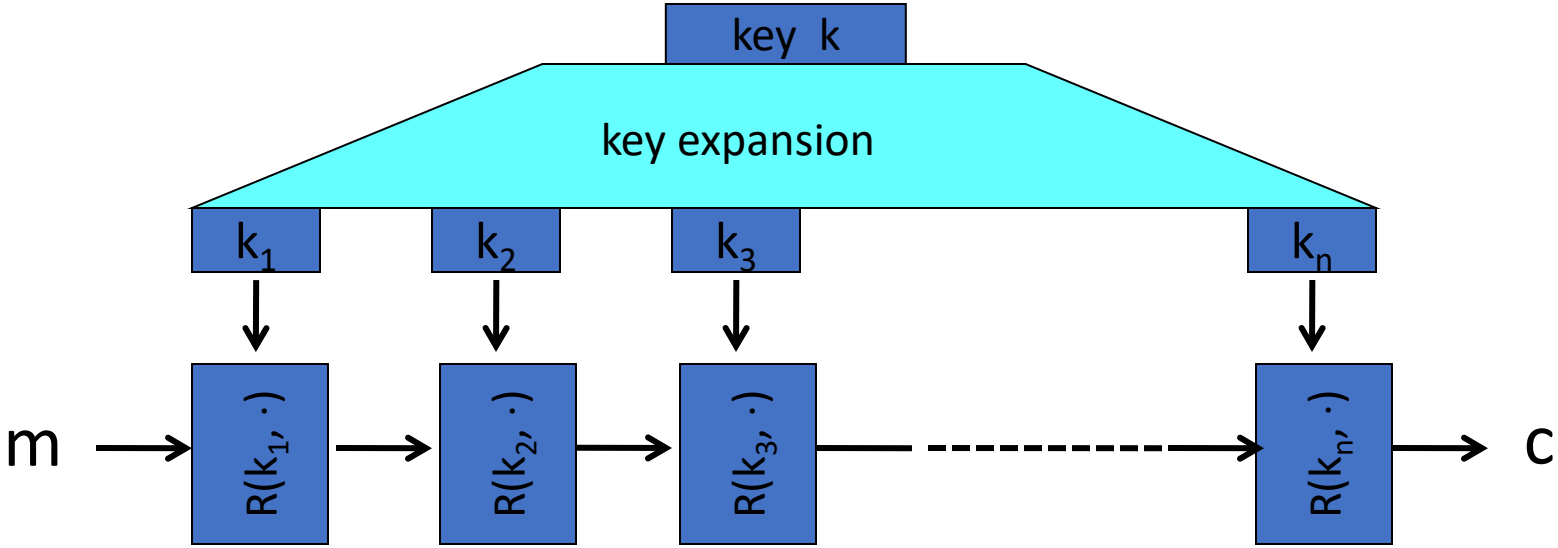
Canonical examples:

1. 3DES: n= 64 bits, k = 168 bits
2. AES: n=128 bits, k = 128, 192, 256 bits

Block Ciphers Built by Iteration

R(k,m) is called a round function

for 3DES (n=48), for AES-128 (n=10)



The Data Encryption Standard (DES)

- Early 1970s: Horst Feistel designs Lucifer at IBM

  key-len = 128 bits ; block-len = 128 bits
- 1973: NBS asks for block cipher proposals.

  IBM submits variant of Lucifer.
- 1976: NBS adopts DES as a federal standard

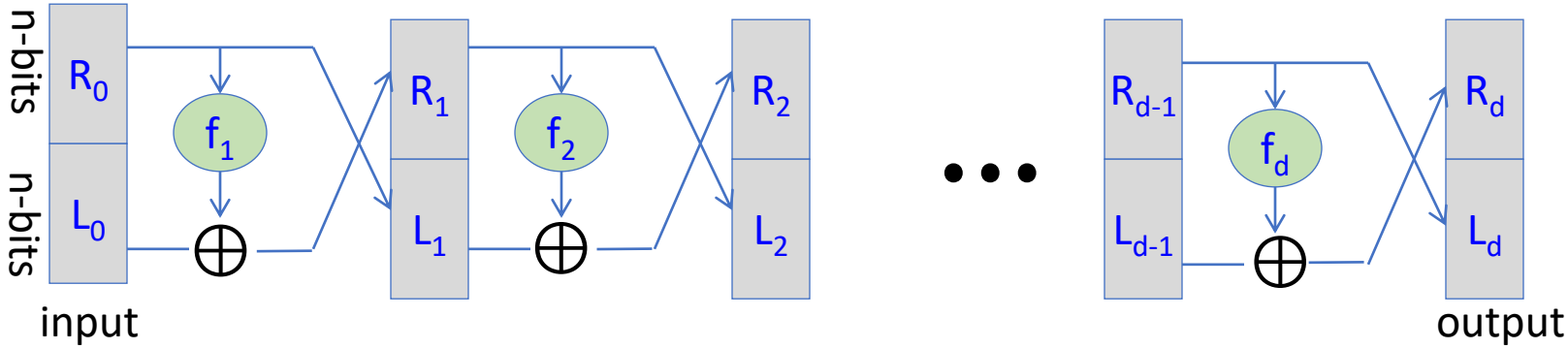
  key-len = 56 bits ; block-len = 64 bits
- 1997: DES broken by exhaustive search
- 2000: NIST adopts Rijndael as AES to replace DES

Widely deployed in banking (ACH) and commerce

DES: core idea – Feistel Network

Đưa ra các hàm $f_1, \ldots, f_d$: $\{0,1\}^n \longrightarrow \{0,1\}^n$

Mục tiêu: thiết lập hàm khả nghịch $F: \{0,1\}^{2n} \longrightarrow \{0,1\}^{2n}$

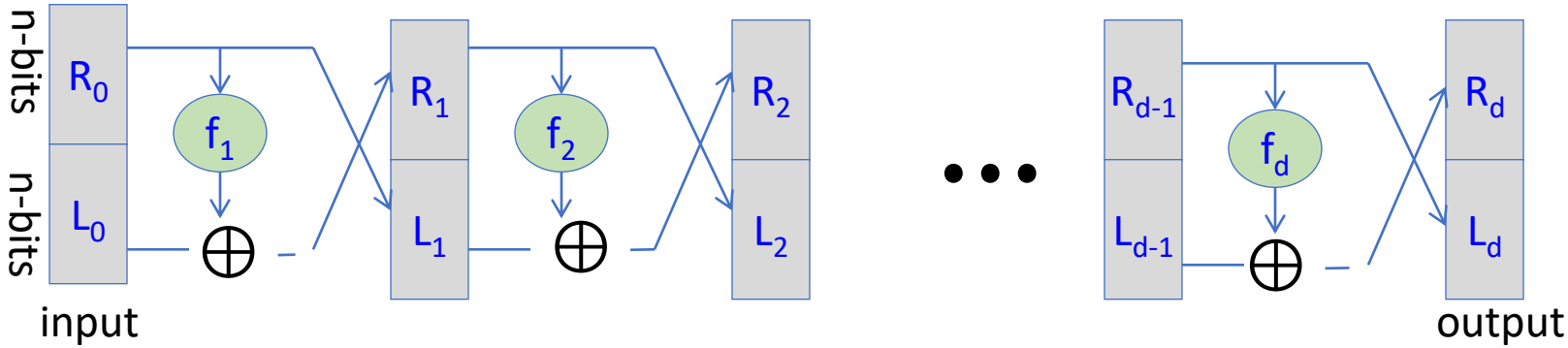


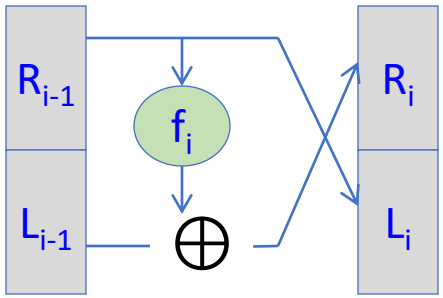
In symbols: $\begin{cases} R_i = F_i(R_{i-1}) \oplus L_{i-1} \\ L_{i-1} = R_{i-1} \end{cases}$

DES: core idea – Feistel Network

Claim: cho tất cả $f_1, \ldots, f_d$: $\{0,1\}^n \longrightarrow \{0,1\}^n$

Feistel network $F: \{0,1\}^{2n} \longrightarrow \{0,1\}^{2n}$ có tính khả nghịch Chứng minh: cấu trúc khả nghịch





inverse

$R_{i-1} = L_i$

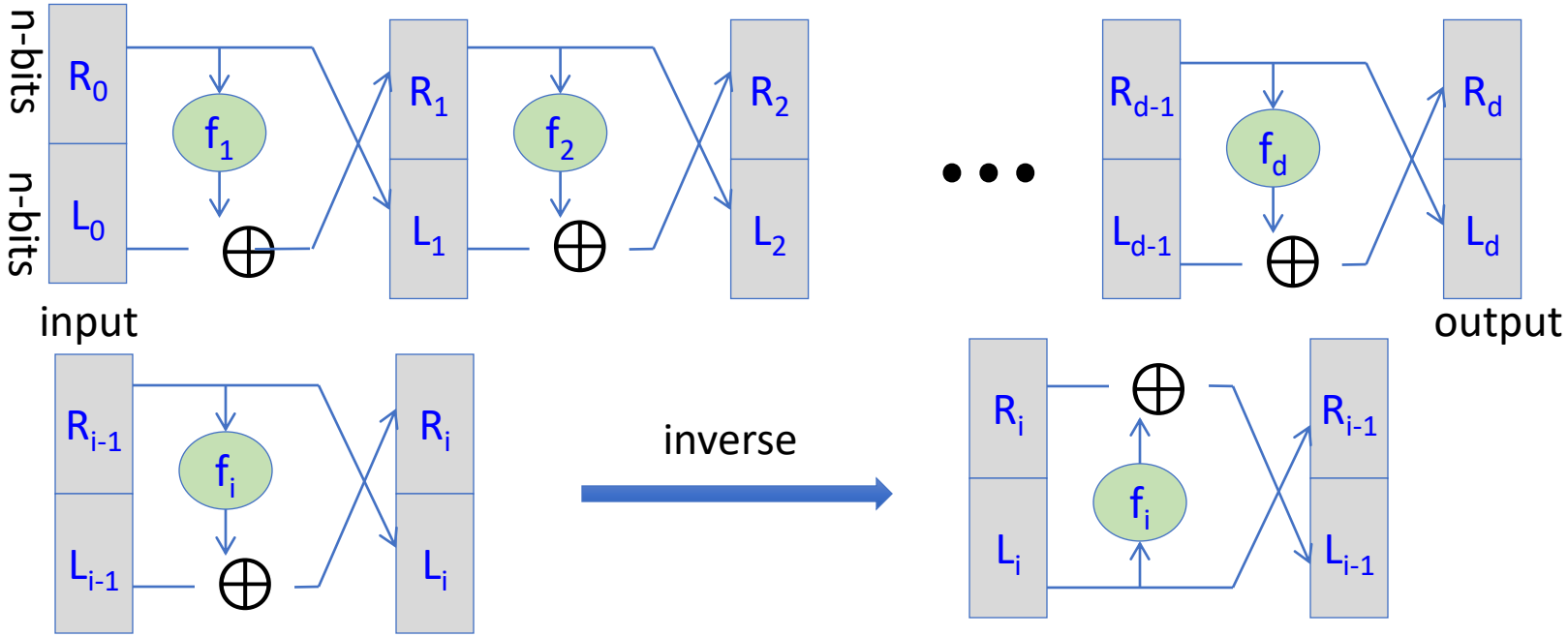
$L_{i-1} =$

DES: core idea – Feistel Network

Claim: for all $f_1, \ldots, f_d$: $\{0,1\}^n \longrightarrow \{0,1\}^n$

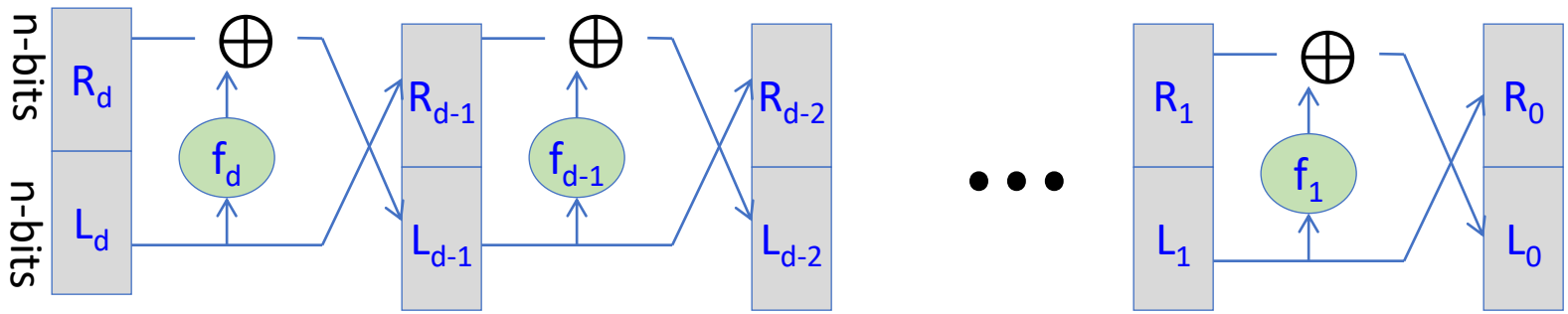
Feistel network $F: \{0,1\}^{2n} \longrightarrow \{0,1\}^{2n}$ is invertible

Proof: construct inverse



Decryption circuit

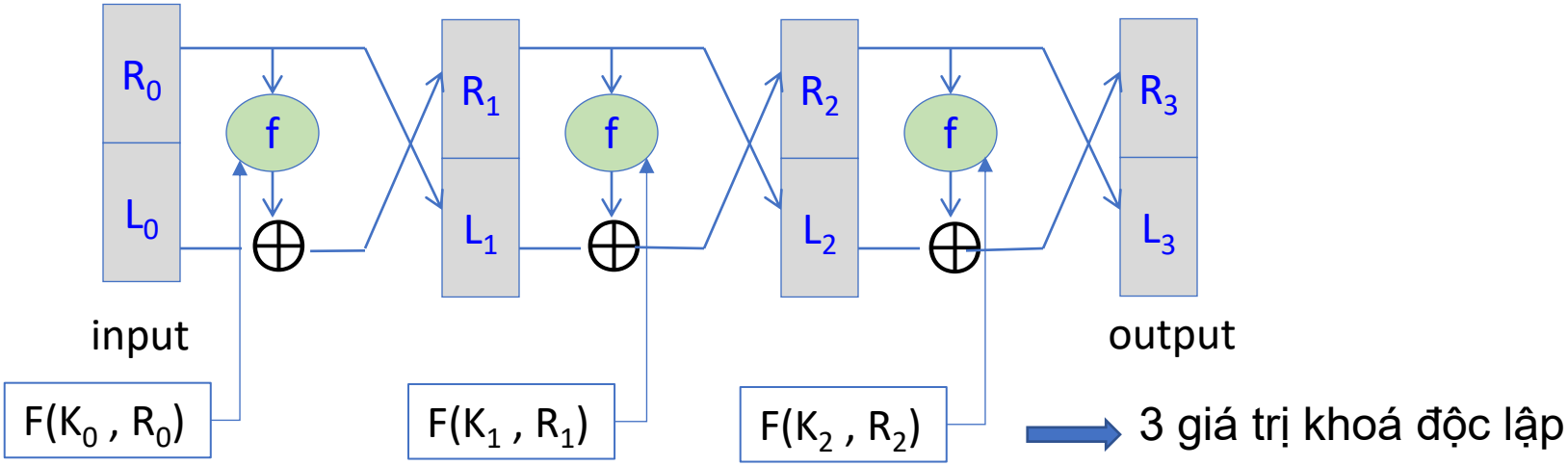
- Nghịch đảo về cơ bản là cùng cấu trúc mạch với $f_1, \ldots, f_d$ áo dụng theo thứ tự ngược lại
- Phương pháp chung để xây dựng các hàm khả nghịch (mật mã khối) từ các hàm tùy ý.
- Sử dụng trong nhiều mật mã khối nhưng không dùng trong AES



“Thm:” (Luby-Rackoff ‘85):

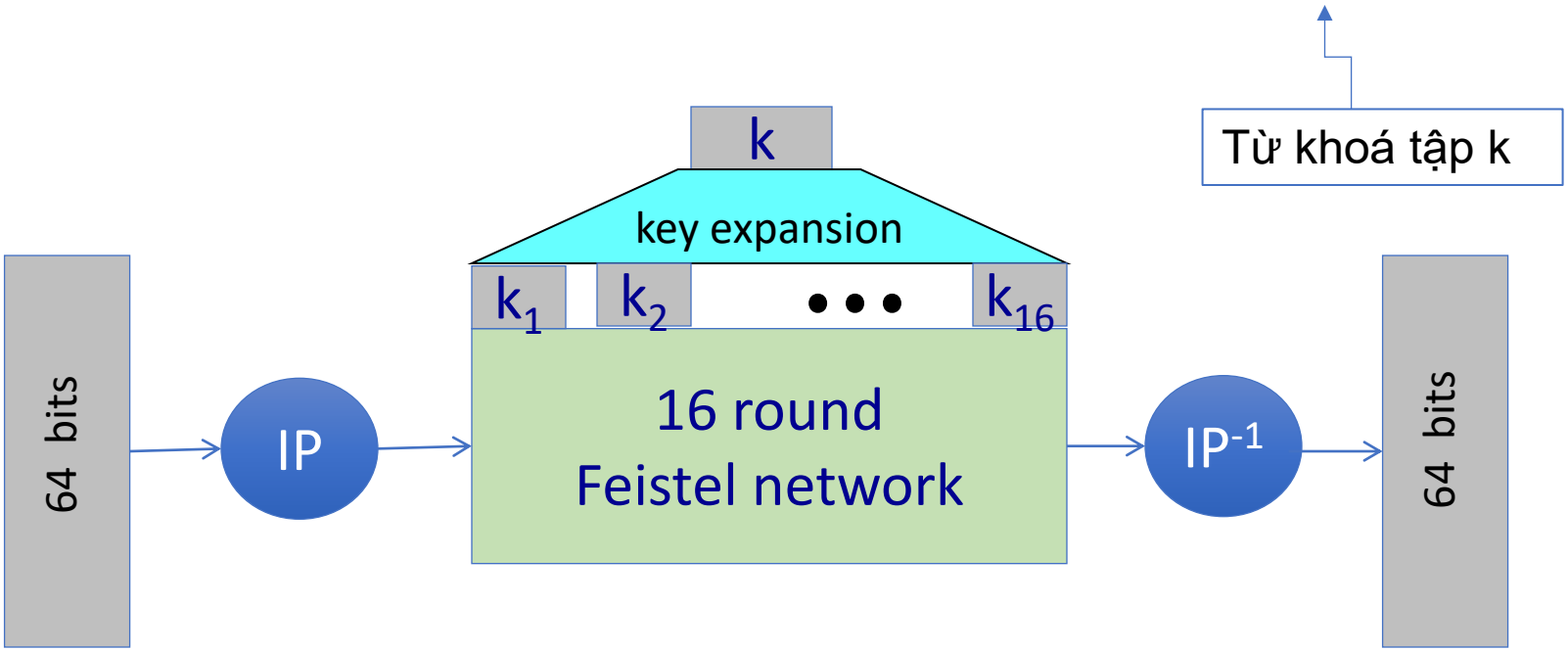
$f: K \times \{0,1\}^n \longrightarrow \{0,1\}^n$ là một PRF bảo mật

$\Rightarrow$ 3-round Feistel $F: K^3 \times \{0,1\}^{2n} \longrightarrow \{0,1\}^{2n}$ là PRP bảo mật



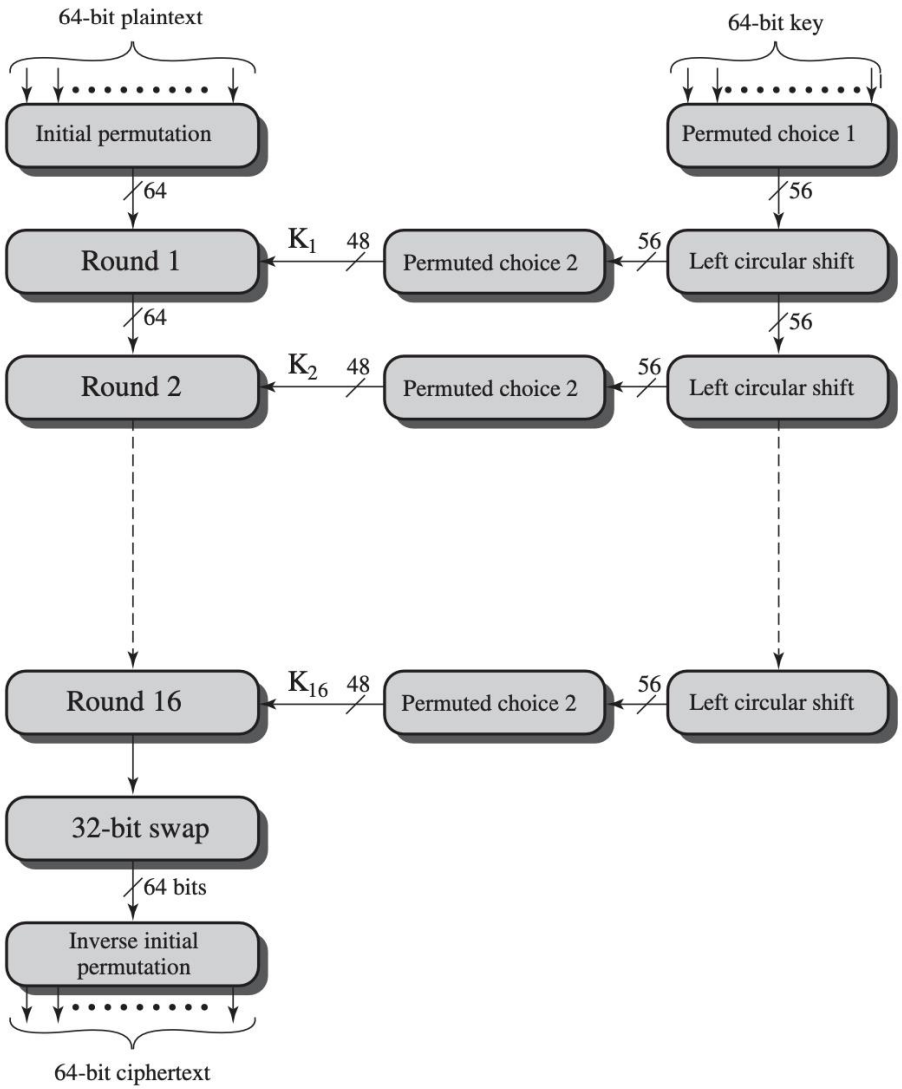
DES: 16 round Feistel network

$$f_1, \ldots, f_{16}: \{0,1\}^{32} \longrightarrow \{0,1\}^{32} \quad , \quad f_i(x) = \mathbf{F}(k_i, x)$$

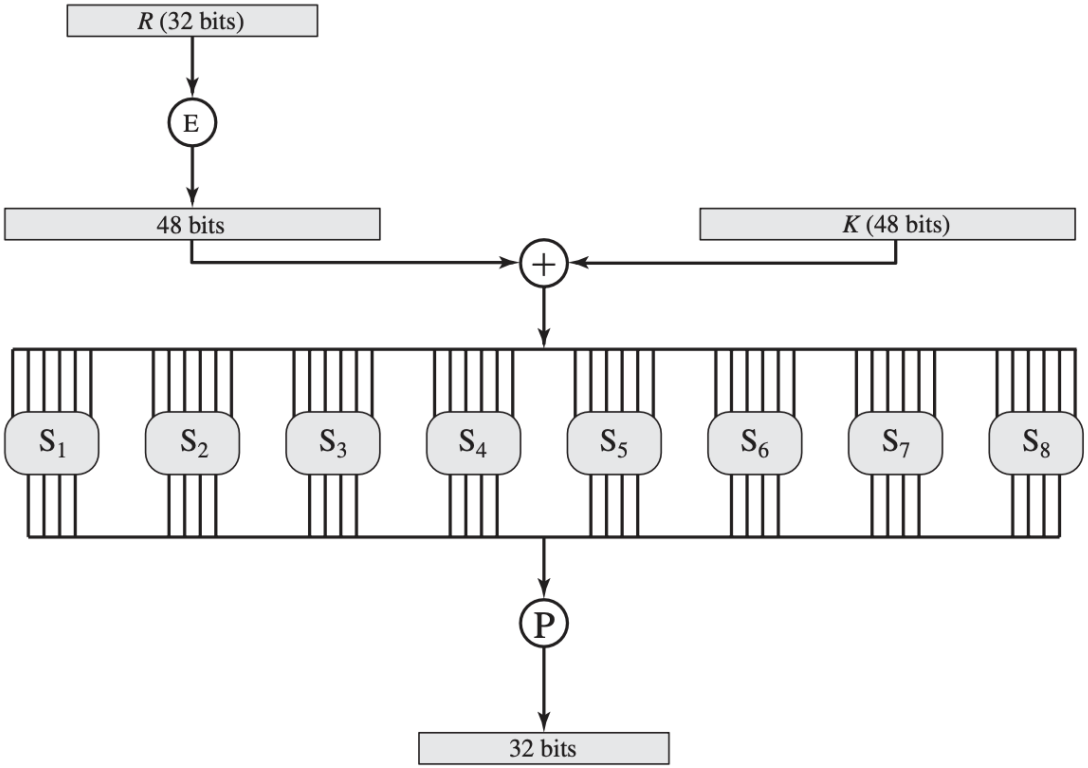


To invert, use keys in reverse order

General Depiction of DES Encryption Algorithm



The function $F(k_i, x)$



S-box: function $\{0,1\}^6 \longrightarrow \{0,1\}^4$, implemented as look-up table.

Permutation Tables for DES

(a) Initial Permutation (IP)

58	50	42	34	26	18	10	2
60	52	44	36	28	20	12	4
62	54	46	38	30	22	14	6
64	56	48	40	32	24	16	8
57	49	41	33	25	17	9	1
59	51	43	35	27	19	11	3
61	53	45	37	29	21	13	5
63	55	47	39	31	23	15	7

(b) Inverse Initial Permutation (IP^{-1})

40	8	48	16	56	24	64	32
39	7	47	15	55	23	63	31
38	6	46	14	54	22	62	30
37	5	45	13	53	21	61	29
36	4	44	12	52	20	60	28
35	3	43	11	51	19	59	27
34	2	42	10	50	18	58	26
33	1	41	9	49	17	57	25

(c) Expansion Permutation (E)

32	1	2	3	4	5
4	5	6	7	8	9
8	9	10	11	12	13
12	13	14	15	16	17
16	17	18	19	20	21
20	21	22	23	24	25
24	25	26	27	28	29
28	29	30	31	32	1

(d) Permutation Function (P)

16	7	20	21	29	12	28	17
1	15	23	26	5	18	31	10
2	8	24	14	32	27	3	9
19	13	30	6	22	11	4	25

$X = (\mathrm{IP}(M))$

M_1	M_2	M_3	M_4	M_5	M_6	M_7	M_8
M_9	M_{10}	M_{11}	M_{12}	M_{13}	M_{14}	M_{15}	M_{16}
M_{17}	M_{18}	M_{19}	M_{20}	M_{21}	M_{22}	M_{23}	M_{24}
M_{25}	M_{26}	M_{27}	M_{28}	M_{29}	M_{30}	M_{31}	M_{32}
M_{33}	M_{34}	M_{35}	M_{36}	M_{37}	M_{38}	M_{39}	M_{40}
M_{41}	M_{42}	M_{43}	M_{44}	M_{45}	M_{46}	M_{47}	M_{48}
M_{49}	M_{50}	M_{51}	M_{52}	M_{53}	M_{54}	M_{55}	M_{56}
M_{57}	M_{58}	M_{59}	M_{60}	M_{61}	M_{62}	M_{63}	M_{64}

M_i is a binary digit

M_{58}	M_{50}	M_{42}	M_{34}	M_{26}	M_{18}	M_{10}	M_2
M_{60}	M_{52}	M_{44}	M_{36}	M_{28}	M_{20}	M_{12}	M_4
M_{62}	M_{54}	M_{46}	M_{38}	M_{30}	M_{22}	M_{14}	M_6
M_{64}	M_{56}	M_{48}	M_{40}	M_{32}	M_{24}	M_{16}	M_8
M_{57}	M_{49}	M_{41}	M_{33}	M_{25}	M_{17}	M_9	M_1
M_{59}	M_{51}	M_{43}	M_{35}	M_{27}	M_{19}	M_{11}	M_3
M_{61}	M_{53}	M_{45}	M_{37}	M_{29}	M_{21}	M_{13}	M_5
M_{63}	M_{55}	M_{47}	M_{39}	M_{31}	M_{23}	M_{15}	M_7

The S-boxes

$$S_i: \{0,1\}^6 \longrightarrow \{0,1\}^4$$

S_5		Middle 4 bits of input															
		0000	**0001**	**0010**	**0011**	**0100**	**0101**	**0110**	**0111**	**1000**	**1001**	**1010**	**1011**	**1100**	**1101**	**1110**	**1111**
Outer bits	**00**	0010	1100	0100	0001	0111	1010	1011	0110	1000	0101	0011	1111	1101	0000	1110	1001
	01	1110	1011	0010	1100	0100	0111	1101	0001	0101	0000	1111	1010	0011	1001	1000	0110
	10	0100	0010	0001	1011	1010	1101	0111	1000	1111	1001	1100	0101	0110	0011	0000	1110
	11	1011	1000	1100	0111	0001	1110	0010	1101	0110	1111	0000	1001	1010	0100	0101	0011

Ví dụ: Một lựa chọn S-box (bad) xấu

Giả sử:

$$S_i(x_1, x_2, \ldots, x_6) = (x_2 \oplus x_3,\quad x_1 \oplus x_4 \oplus x_5,\quad x_1 \oplus x_6,\quad x_2 \oplus x_3 \oplus x_6)$$

Phương trình tương đương: $S_i(\mathbf{x}) = A_i \cdot \mathbf{x} \pmod 2$

$$\begin{bmatrix} 0&1&1&0&0&0 \\ 1&0&0&1&1&0 \\ 1&0&0&0&0&1 \\ 0&1&1&0&0&1 \end{bmatrix} \cdot \begin{bmatrix} x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \\ x_6 \end{bmatrix} = \begin{bmatrix} x_2 \oplus x_3 \\ x_1 \oplus x_4 \oplus x_5 \\ x_1 \oplus x_6 \\ x_2 \oplus x_3 \oplus x_6 \end{bmatrix}$$

Có thể khẳng định S_i là một hàm tuyến tính.

Example: a bad S-box choice

Khi các thực thể bên trong mật mã DES là tuyến tính:

Tồn tại một ma trận nhị phân B sao cho:

$$\mathrm{DES}(k,m) = \overset{832}{64\,\boxed{B}} \cdot \begin{bmatrix} m \\ k_1 \\ k_2 \\ \vdots \\ k_{16} \end{bmatrix} = \boxed{c} \pmod 2$$

$$K = \begin{bmatrix} k_1 \\ k_2 \\ \vdots \\ k_{16} \end{bmatrix}$$

Do vậy: $\mathrm{DES}(k,m_1) \oplus \mathrm{DES}(k,m_2) \oplus \mathrm{DES}(k,m_3)$

$= \mathrm{DES}(k, m_1 \oplus m_2 \oplus m_3)$

$$B \begin{bmatrix} m_1 \\ k \end{bmatrix} \oplus B \begin{bmatrix} m_2 \\ k \end{bmatrix} \oplus B \begin{bmatrix} m_3 \\ k \end{bmatrix} = B \begin{bmatrix} m_1 \oplus m_2 \oplus m_3 \\ k \oplus k \oplus k \end{bmatrix}$$

Lựa chọn S-boxes và P-box

- Việc lựa chọn S-boxes and P-box không theo quy tắc sẽ tạo ra các mã khối không bảo mật
- (key recovery after $\approx 2^{24}$ outputs) [BS'89]
- Một số quy tắc để lựa chọn S và P boxes:
- Quan hệ bit đầu ra và đầu vào không phải qua một 1 hàm tuyến tính
- S-boxes là ánh xạ 4-to-1 …

Tìm kiếm vét cạn (Exhaustive Search) đối với khoá

Mục tiêu: đưa ra một số cặp đầu vào đầu ra (m_i, c_i = E(k, m_i)) i=1,..,3 tìm khoá k.

(m_1 , m_2 , m_3) -> (c_1 ,c_2 , c_3) tìm k

Định lý: Giả sử DES là một mã mật lý tưởng (***ideal cipher)*** (2^{56} hàm khả nghịch ngẫu nhiên π_1 π_2... $\pi_{2^{56}}$: $[0,1]^{64} \rightarrow [0,1]^{64}$) khi đó ∀ m, c sẽ tồn tại một khoá k sao cho c = DES(k, m) với xác suất $\text{prob.} \geq 1 - 1/256 \approx 99.5\%$.

Proof: $\Pr[\exists\, k' \neq k\colon\ C = \mathrm{DES}(k,m) = \mathrm{DES}(k',m)] \leq \sum_{k' \in [0,1]^{56}} \Pr[\mathrm{DES}(k,m) = \mathrm{DES}(k',m)] \leq 2^{56}.1/\ 2^{64} = 1/\ 2^{8}$

Tìm kiếm vét cạn khoá đối của mật mã khối

Cho 2 cặp DES $(m_1, c_1=DES(k, m_1))$, $(m_2, c_2=DES(k, m_2))$, xác xuất đảm bảo tính duy nhất (unicity prob.) ≈ $1 - 1/2^{71}$

Mã AES-128: đưa ra 2 cặp khoá (inp/out pairs), xác xuất đảm bảo tính duy nhất (unicity prob.) ≈ $1 - 1/2^{128}$

⇒ hai cặp input/output pairs của AES là đủ lớn cho tấn công vét cạn khoá

DES challenge

msg = "The unknown messages is: XXXX … "

CT = c_1 c_2 c_3 c_4

Goal: find $k \in \{0,1\}^{56}$ s.t. $DES(k, m_i) = c_i$ for $i=1,2,3$

1997: Internet search -- **3 months**

1998: EFF machine (deep crack) -- **3 days** (250K $)

1999: combined search -- **22 hours**

2006: COPACOBANA (120 FPGAs) -- **7 days** (10K $)

⇒ 56-bit ciphers là độ dài không đủ mạnh!!!

Độ mạnh của DES chống lại tấn công vét cạn

Phương pháp 1: Thử với **Triple-DES**

- Cho $E : K \times M \longrightarrow M$ là một hệ mật mã khối
- Định nghĩa **3E**: $K^3 \times M \longrightarrow M$ như sau

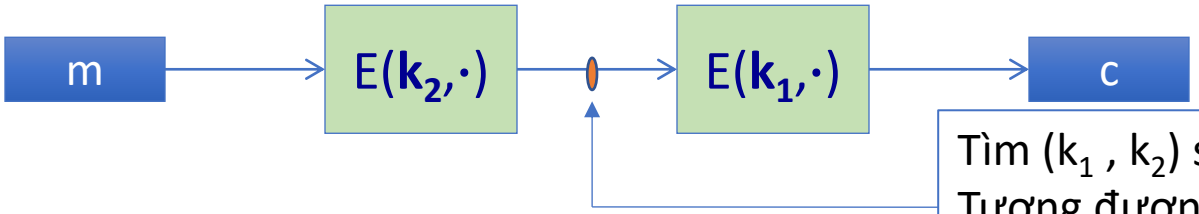
$$\mathbf{3E}\big((k_1,k_2,k_3), m\big) = E(k_1, D(k_2,E(k_3,m)))$$

$k_1 = k_2 = k_3$ -> Single DES

- Cho hệ mã 3DES: key-size = 3×56 = 168 bits. 3DES chậm hơn DES.
- Tấn công vét cạn 128 bit – key là 2^{72} ngày.
- Tấn công đơn giản 3DES trong thời gian $\approx 2^{118}$ ngày.

Tại sao không phải là double DES?

- Định nghĩa $2E((k_1,k_2), m) = E(k_1, E(k_2, m))$



key-len = 112 bits for DES

Tìm (k_1, k_2) sao cho $E(k_1, E(k_2, m)) = c$.
Tương đương:
$E(k_2, m) = D(k_1, c)$

Tấn công thiết lập 10 cặp: $M = (m_1,\ldots, m_{10})$, $C = (c_1,\ldots,c_{10})$.

- Bước 1: Xây bảng

tham chiếu liệt kê cột thứ 2

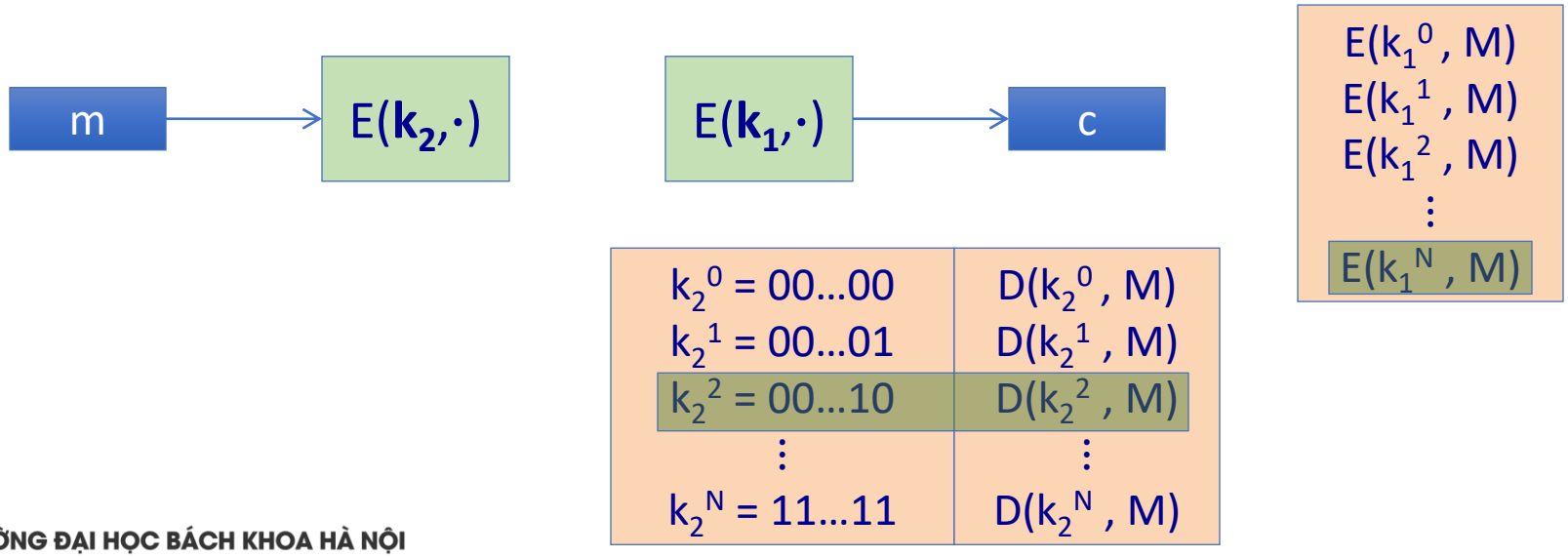
$k_1^0 = 00\ldots00$	$E(k_1^0, M)$
$k_1^1 = 00\ldots01$	$E(k_1^1, M)$
$k_1^2 = 00\ldots10$	$E(k_1^2, M)$
⋮	⋮
$k_1^N = 11\ldots11$	$E(k_1^N, M)$

2^{56} entries

Tấn công xen giữa (Meet in the middle attack)

Tấn công thiết lập 10 cặp : $M = (m_1,\ldots, m_{10})$, $C = (c_1,\ldots,c_{10})$

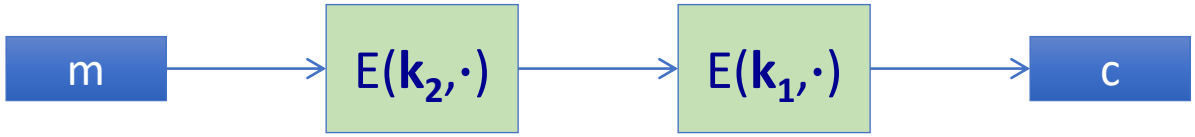
- step 1: build table.
- Step 2: với tất cả $k \in \{0,1\}^{56}$ thực hiện kiểm tra nếu $D(k, C)$ ở trong cột thứ 2nd. Thì ta có $E(k^i,M) = D(k,C)$ $\Rightarrow$ $(k^i,k) = (k_2,k_1)$



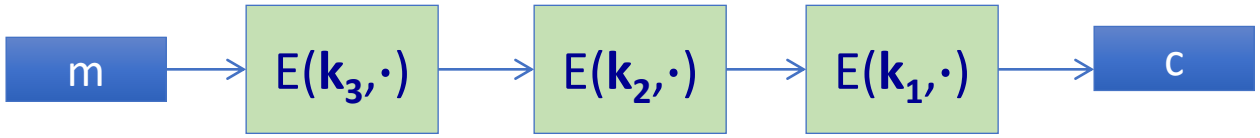
Meet in the middle attack

$$\text{Time} = \underbrace{2^{56}\log(2^{56})}_{\text{Build + sort table}} + \underbrace{2^{56}\log(2^{56})}_{\text{Search in table}} < 2^{63} \quad << \quad 2^{112} \ ,$$

$$\text{space} \approx 2^{56}$$



Tương tự với 3DES: $\text{Time} = 2^{118}$, $\text{space} \approx 2^{56}$



Phương pháp 2: DESX

$E : K \times \{0,1\}^n \longrightarrow \{0,1\}^n$ là một hệ mật mã khối

Định nghĩa EX như sau: $EX((k_1,k_2,k_3), m) = k_1 \oplus E(k_2, m \oplus k_3)$

Với DESX: key-len = 64+56+64 = 184 bits

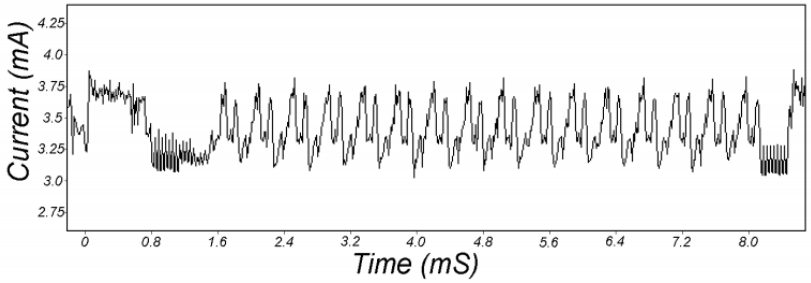
Tấn công trong thời gian $2^{64+56} = 2^{120}$ (homework)

Note: $k_1 \oplus E(k_2, m)$ and $E(k_2, m \oplus k_1)$ does nothing !!

Các tấn công khi thực thi

1. Tấn công kênh kề (Side channel attacks):
   - Đo thời gian (**time)** mã hoá và giải mã (enc/dec), đo năng lượng tiêu thụ cho mã hoá và giải mã

smartcard



2. Tấn công lỗi (Fault attacks): [Kocher, Jaffe, Jun, 1998]
   - Tính toán lỗi ở vòng cuối cùng để tìm ra khoá mật K

⇒ do not even implement crypto primitives yourself

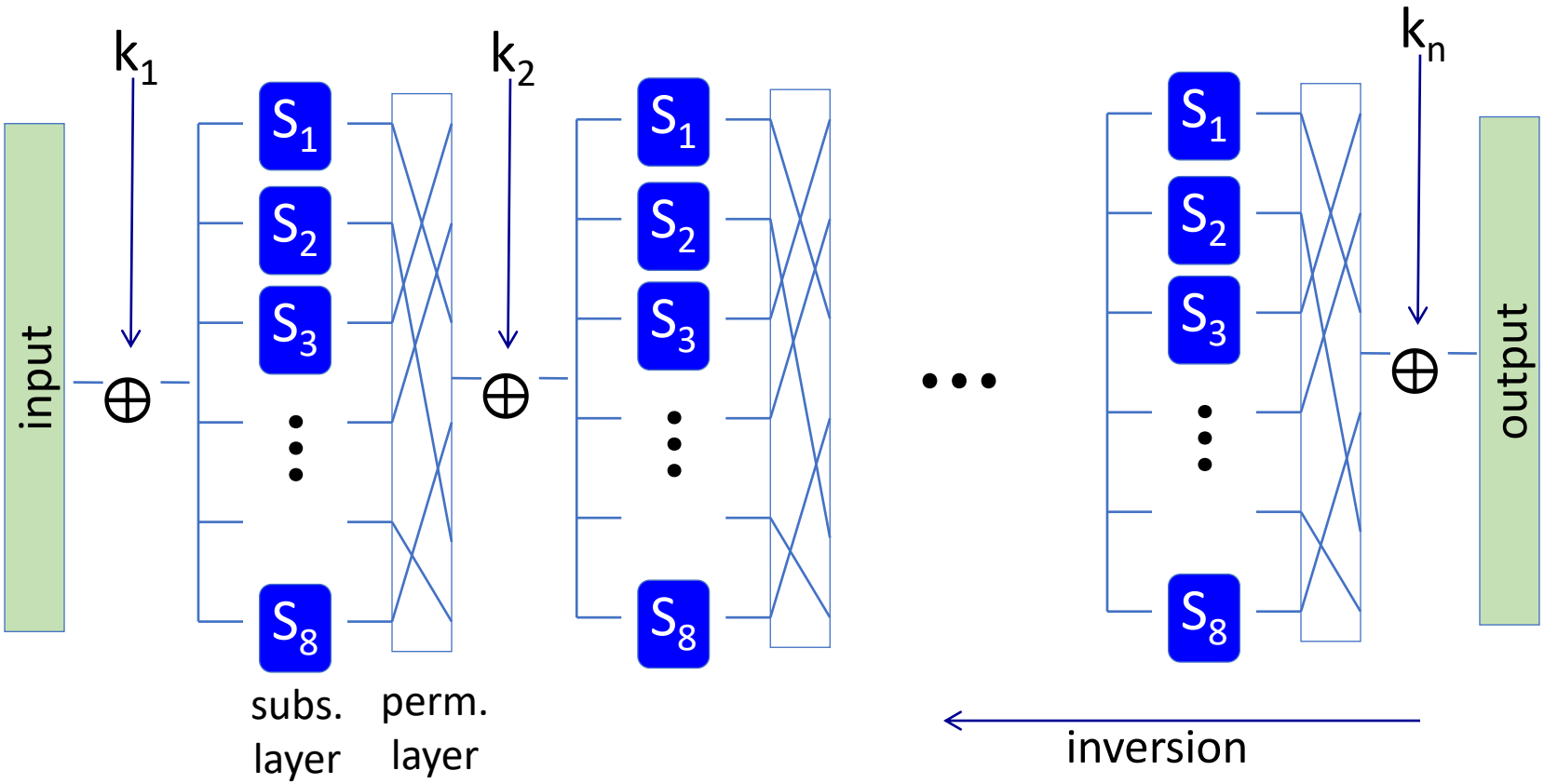
…

Quá trình phát triển AES

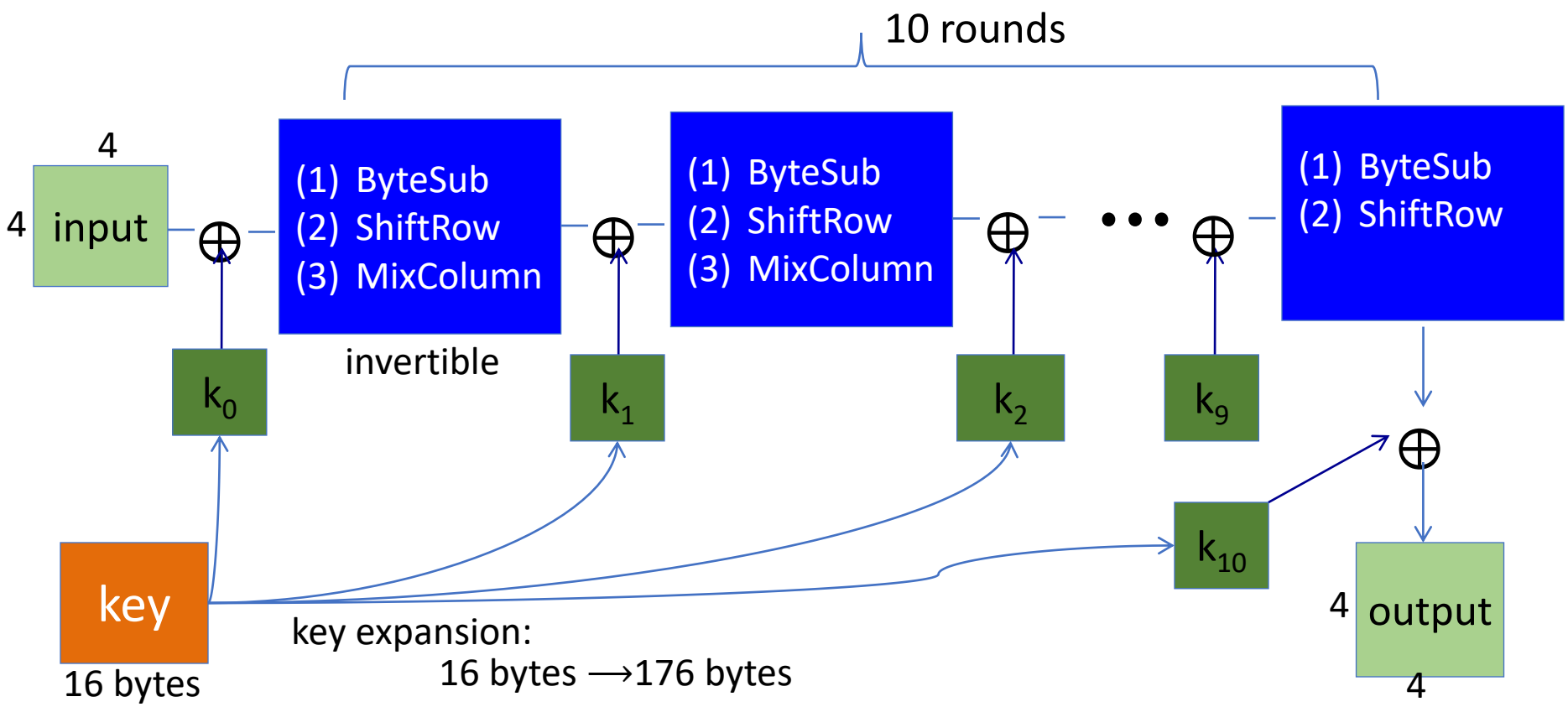
- 1997: NIST publishes request for proposal
- 1998: 15 submissions. Five claimed attacks.
- 1999: NIST chooses 5 finalists
- 2000: NIST chooses Rijndael as AES (designed in Belgium)

Key sizes: 128, 192, 256 bits. Block size: 128 bits

AES is a Subs-Perm network (not Feistel)



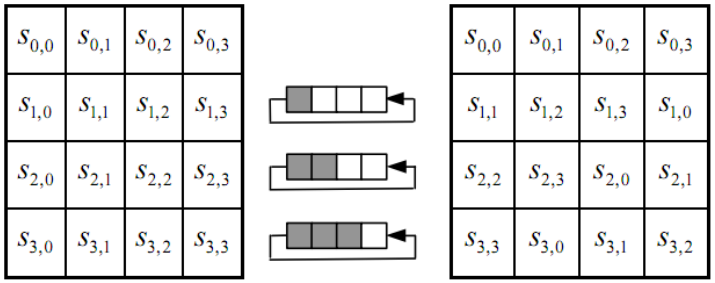
AES-128 schematic



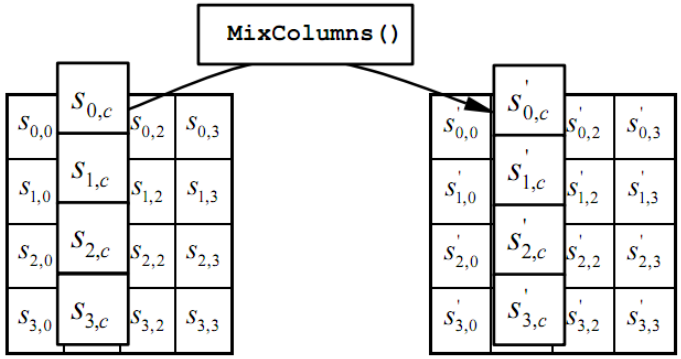
The round function

- **ByteSub:** a 1 byte S-box. 256 byte table (easily computable)

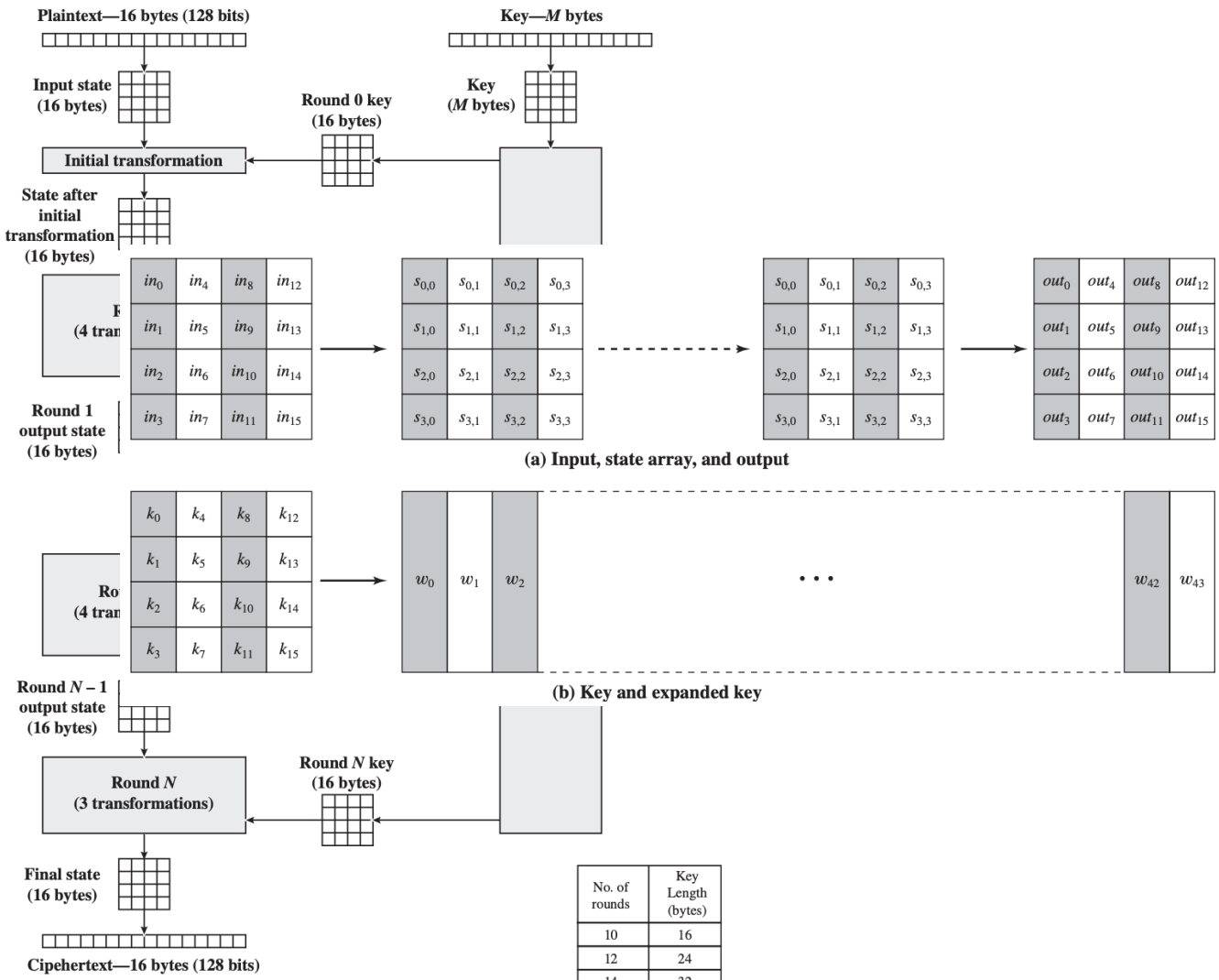
- **ShiftRows:**



- **MixColumns**



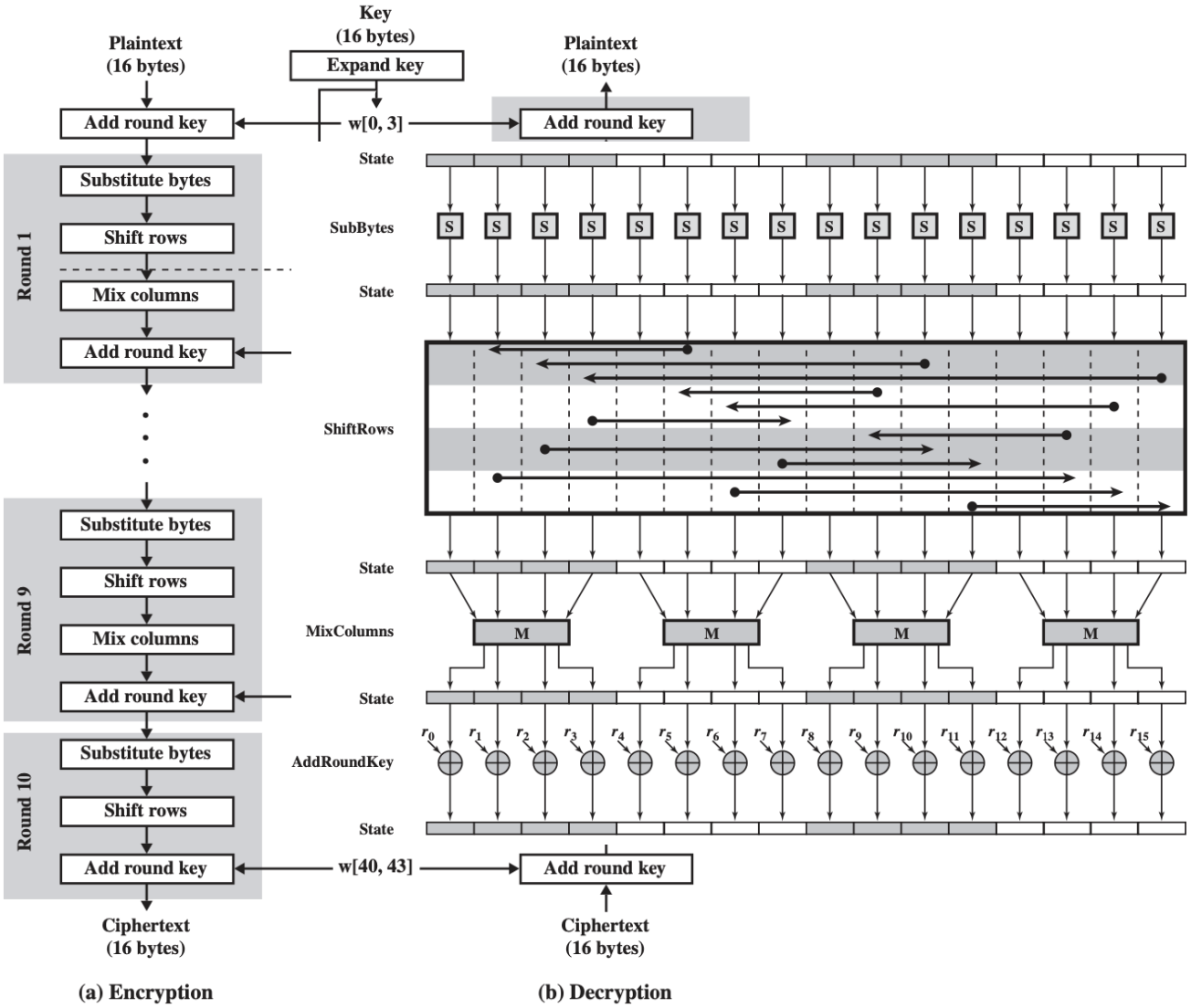
Phân tích chi tiết thuật toán AES



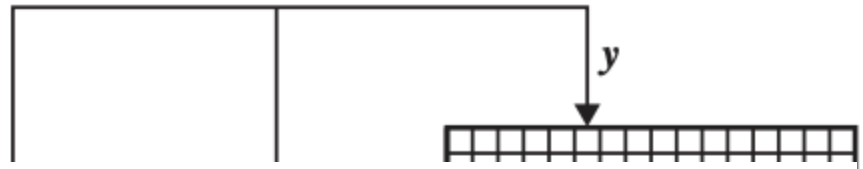
No. of rounds	Key Length (bytes)
10	16
12	24
14	32

Figure 5.1 AES Encryption Process

Phân tích chi tiết thuật toán AES



Substitute Bytes Transformation

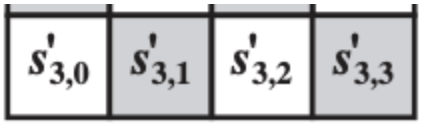


x \ y	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	63	7C	77	7B	F2	6B	6F	C5	30	01	67	2B	FE	D7	AB	76
1	CA	82	C9	7D	FA	59	47	F0	AD	D4	A2	AF	9C	A4	72	C0
2	B7	FD	93	26	36	3F	F7	CC	34	A5	E5	F1	71	D8	31	15
3	04	C7	23	C3	18	96	05	9A	07	12	80	E2	EB	27	B2	75
4	09	83	2C	1A	1B	6E	5A	A0	52	3B	D6	B3	29	E3	2F	84
5	53	D1	00	ED	20	FC	B1	5B	6A	CB	BE	39	4A	4C	58	CF
6	D0	EF	AA	FB	43	4D	33	85	45	F9	02	7F	50	3C	9F	A8
7	51	A3	40	8F	92	9D	38	F5	BC	B6	DA	21	10	FF	F3	D2
8	CD	0C	13	EC	5F	97	44	17	C4	A7	7E	3D	64	5D	19	73
9	60	81	4F	DC	22	2A	90	88	46	EE	B8	14	DE	5E	0B	DB
A	E0	32	3A	0A	49	06	24	5C	C2	D3	AC	62	91	95	E4	79
B	E7	C8	37	6D	8D	D5	4E	A9	6C	56	F4	EA	65	7A	AE	08
C	BA	78	25	2E	1C	A6	B4	C6	E8	DD	74	1F	4B	BD	8B	8A
D	70	3E	B5	66	48	03	F6	0E	61	35	57	B9	86	C1	1D	9E
E	E1	F8	98	11	69	D9	8E	94	9B	1E	87	E9	CE	55	28	DF
F	8C	A1	89	0D	BF	E6	42	68	41	99	2D	0F	B0	54	BB	16

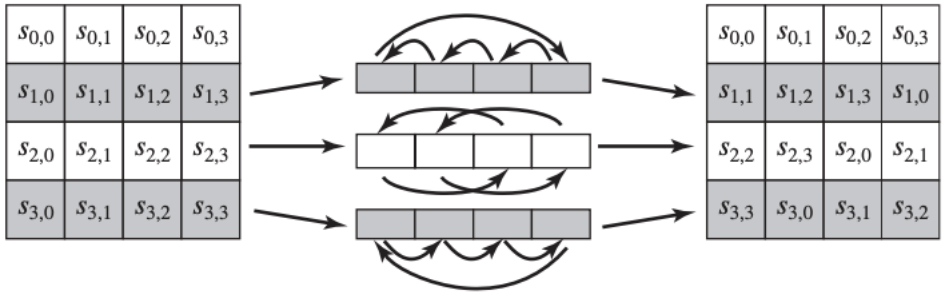
EA	04	65	85
83	45	5D	96
5C	33	98	B0
F0	2D	AD	C5

→

87	F2	4D	97
EC	6E	4C	90
4A	C3	46	E7
8C	D8	95	A6



ShiftRows và MixColumn Transformation



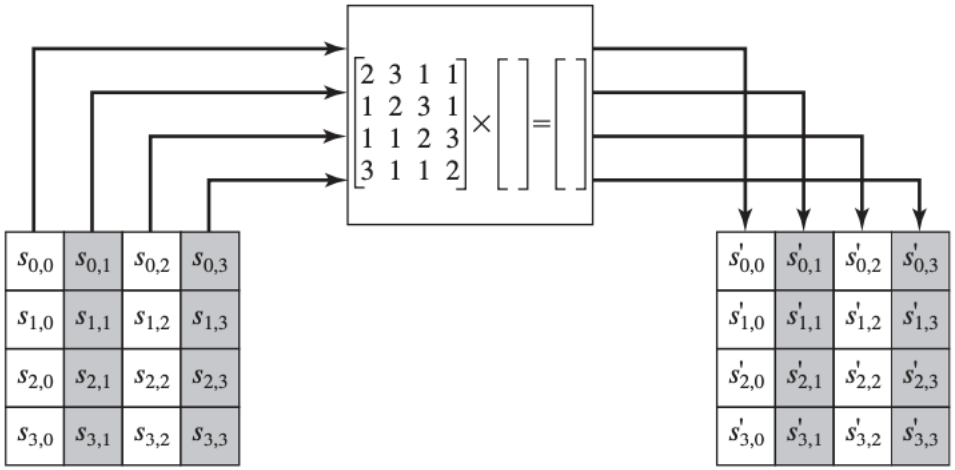
(a) Shift row transformation

87	F2	4D	97
EC	6E	4C	90
4A	C3	46	E7
8C	D8	95	A6

→

87	F2	4D	97
6E	4C	90	EC
46	E7	4A	C3
A6	8C	D8	95

Dịch hàng



(b) Mix column transformation

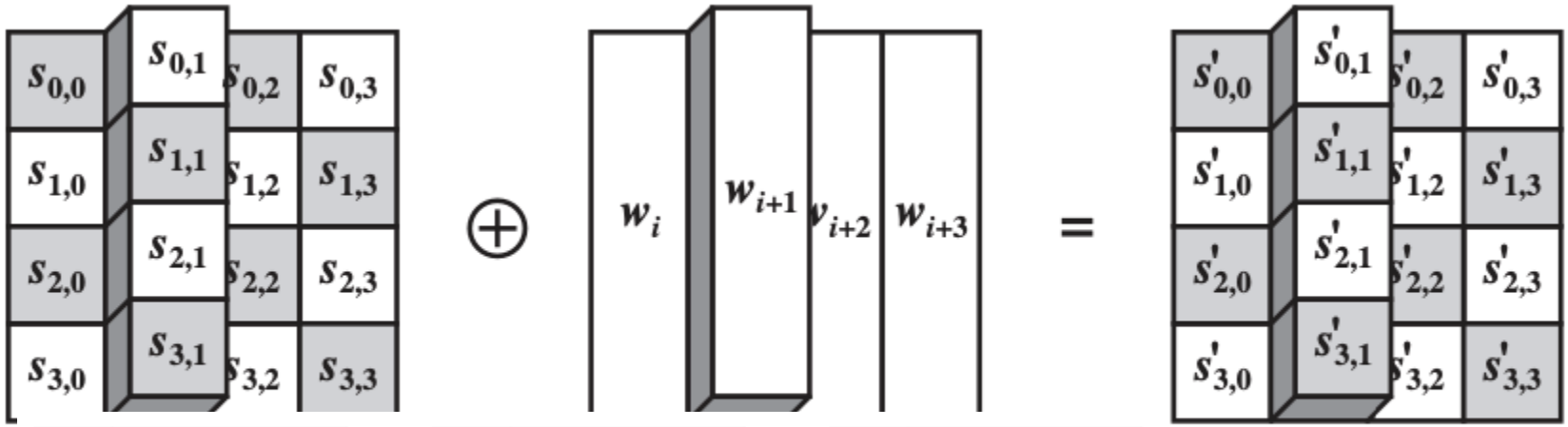
87	F2	4D	97
6E	4C	90	EC
46	E7	4A	C3
A6	8C	D8	95

→

47	40	A3	4C
37	D4	70	9F
94	E4	3A	42
ED	A5	A6	BC

Trộn cột

AddRoundKey Transformation



47	40	A3	4C
37	D4	70	9F
94	E4	3A	42
ED	A5	A6	BC

$\oplus$

AC	19	28	57
77	FA	D1	5C
66	DC	29	00
F3	21	41	6A

=

EB	59	8B	1B
40	2E	A1	C3
F2	38	13	42
1E	84	E7	D6

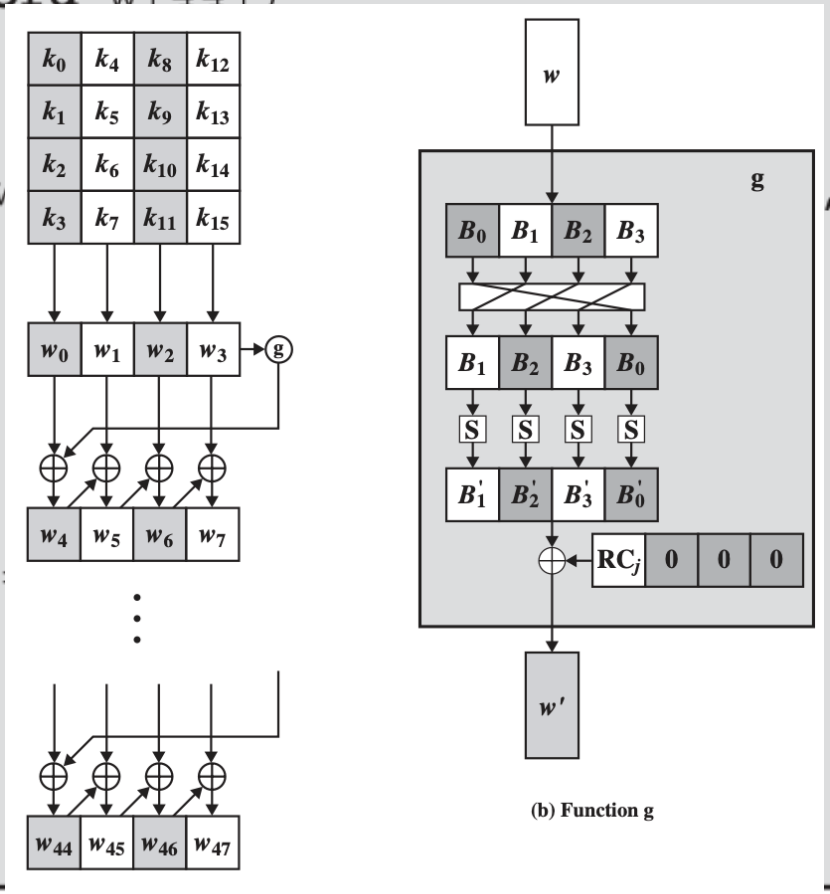
AES key expansion

```
KeyExpansion (byte key[16], word w[44])
{
    word temp
    for (i = 0; i < 4; i++)    w

    for (i = 4; i < 44; i++)
    {
     temp = w[i - 1];
     if (i mod 4 = 0)    temp

     w[i] = w[i-4] ⊕ temp
}
```

j	1	2	3	4	5	6	7	8	9	10
RC[j]	01	02	04	08	10	20	40	80	1B	36



(a) Overall algorithm

(b) Function g

AES example

Plaintext:	0123456789abcdeffedcba9876543210
Key:	0f1571c947d9e8590cb7add6af7f6798
Ciphertext:	ff0b844a0853bf7c6934ab4364148fb9

Code size/performance tradeoff

	Code size	Performance
Pre-compute round functions (24KB or 4KB)	largest	fastest: table lookups and xors
Pre-compute S-box only (256 bytes)	smaller	slower
No pre-computation	smallest	slowest